

THE ML / DL HANDBOOK — COMPANION SERIES

Volume V

Deep Learning

Foundations

The companion deep-dive to Part V of the ML/DL Handbook — the neuron, backpropagation, the optimizer family, and the vanishing-gradient story that quietly explains most of modern deep learning.

Chapter 14 — The Neuron, and Why Depth Needs a Kink

Chapter 15 — Backpropagation, and the Vanishing Signal

Chapter 16 — The Optimizer Family, One Fix at a Time

Chapter 17 — Regularizing Deep Nets, Three Pillars

July 17, 2026

How to Read This Volume

This is one of your two highest-probability interview areas. The goal is to make back-propagation, the optimizer family, and the vanishing-gradient story feel inevitable — so that in the interview you are reconstructing ideas from intuition, not reciting them.

The one question this volume keeps answering. Nearly every technique in these four chapters is a response to a single problem: a gradient signal, multiplied layer after layer, either dies or explodes. ReLU, He initialization, BatchNorm, residual connections, LSTM gates, gradient clipping, even Adam — each is an answer to some version of it. When you meet a new trick, ask: *which part of the multiplicative chain is this protecting?*

Each chapter runs intuition first, then the mathematics that makes the intuition precise. The *Derivation* panels are the part worth doing with a pen: they show every intermediate step, because the interview question is rarely *state the formula* and almost always *why does this term appear?*

The figures are not schematics. Where a claim can be measured — the gradient decaying through 25 layers, four optimizers on the same ravine, BatchNorm pinning each layer's activations to a healthy band — the plot is real code on real numbers, and the caption reports what actually happened. Where the two disagree, trust the measurement.

Worth Knowing boxes add practitioner context the source material assumes you'll pick up on the job. *Interview Question* boxes point back at the section that already answered them.

Chapter 14 — The Neuron, and Why Depth Needs a Kink	4
14.1 A neuron is a tiny decision-maker	4
14.2 Why you cannot skip the kink	5
14.3 The activation function's evolution	7
14.4 Chapter 14 in one breath	11
 Chapter 15 — Backpropagation, and the Vanishing Signal	 12
15.1 The blame-assignment problem	12
15.2 The chain of responsibility	13
15.3 Why the signal vanishes	15
15.4 Initialization: the other half of the fix	17
15.5 The whole zoo of fixes traces back to one problem	19
15.6 Chapter 15 in one breath	21
 Chapter 16 — The Optimizer Family, One Fix at a Time	 22
16.1 A family tree, not a list	22
16.2 SGD — the hiker with one problem	22
16.3 Momentum — give the hiker inertia	25
16.4 AdaGrad — a custom step size per parameter	26
16.5 RMSProp — forget the ancient past	27
16.6 Adam — combine the two best ideas	27
16.7 Learning-rate schedules and the modern footnote	30
16.8 Chapter 16 in one breath	32
 Chapter 17 — Regularizing Deep Nets, Three Pillars	 33
17.1 Deep nets overfit spectacularly — three defenses	33
17.2 Dropout — forcing redundancy	33
17.3 Batch normalization — smoothing the terrain	35
17.4 Residual connections — the gradient highway	37
17.5 Chapter 17 in one breath	41

CHAPTER 14

The Neuron, and Why Depth Needs a Kink

14.1 A neuron is a tiny decision-maker

Strip a neuron to its essence and it is doing something you already understand: it takes inputs, weighs each by importance, adds them up, and then decides how strongly to “fire.”

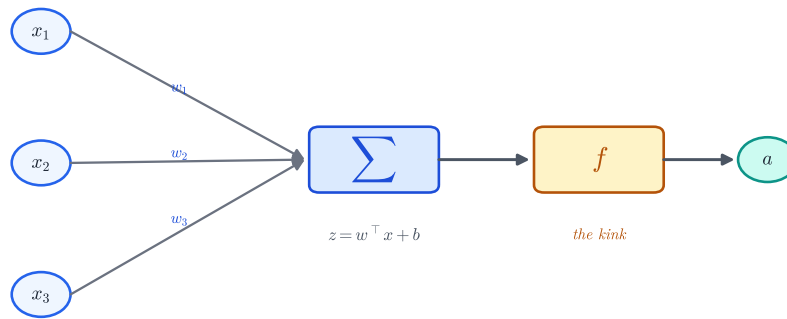


Figure 14.1: A neuron is Volume II’s linear model with a nonlinear hat on top. Everything interesting about depth comes from that hat — remove it and the whole network collapses, as §14.2 shows.

Formally, for input $x \in \mathbb{R}^d$, weights $w \in \mathbb{R}^d$, and bias $b \in \mathbb{R}$, a single neuron computes a weighted sum and passes it through an activation f :

$$z = w^\top x + b = \sum_{j=1}^d w_j x_j + b, \quad a = f(z)$$

The weighted sum $w^\top x + b$ is exactly the linear model from Volume II. So a single neuron is *linear regression wearing a hat* (the activation f). The magic is not in one neuron — it is in stacking layers of them. But stacking only works because of that hat.

A layer just does this for many neurons at once. With $W^{(l)}$ collecting one neuron’s weights per column, the layer maps the previous activation vector to the next:

$$z^{(l)} = (W^{(l)})^\top a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

where f is applied elementwise. That pair of equations, repeated L times, is the entire forward pass of a feedforward network — nothing else is happening.

WORTH KNOWING — THE BIAS IS NOT OPTIONAL

Without b , every decision boundary is forced through the origin — the neuron can choose an orientation but not an offset. In practice frameworks fold the bias into the weight matrix by appending a constant-1 feature, which is why you will see $w^T x$ with no explicit bias in many papers. One place it genuinely disappears: a linear layer immediately followed by BatchNorm (§17.3), whose recentering subtracts any constant the bias could add. PyTorch users write `nn.Linear(..., bias=False)` there, and it is a nice detail to notice out loud.

14.2 Why you cannot skip the kink

Here is a fact that surprises people and is a favorite interview question: *if you remove the activation functions, a deep network collapses into a single linear layer, no matter how deep it is.*

The reason is simple algebra. Stacking linear transformations gives another linear transformation: $W_2(W_1x) = (W_2W_1)x$, and W_2W_1 is just some other matrix. A hundred stacked linear layers equal one linear layer. All that depth — wasted, reduced to a single straight-line model.

DERIVATION — WHY DEPTH COLLAPSES WITHOUT A NONLINEARITY

Take $f(z) = z$ at every layer and simply substitute each layer's output into the next.

Matrix multiplication distributes over the sum. Note what each bracket is: a matrix times x , plus a vector.

Step 1 — write the forward pass of an L -layer network with $f = \text{identity}$

$$a^{(L)} = W^{(L)}(W^{(L-1)}(\dots (W^{(1)}x + b^{(1)}) \dots) + b^{(L-1)}) + b^{(L)}$$

Assume it holds for $L-1$ layers, then apply this step once more. The product of matrices is a matrix; the mess of biases is a vector.

Step 2 — expand the two-layer case to see the pattern

$$W^{(2)}(W^{(1)}x + b^{(1)}) + b^{(2)} = (W^{(2)}W^{(1)})x + (W^{(2)}b^{(1)} + b^{(2)})$$

Step 3 — induct on depth

$$a^{(L)} = Wx + b, \quad \text{where } W \text{ is the product of all the layer matrices:}$$

$$a^{(L)} = Wx + b, \quad W = \prod_{l=1}^L W^{(l)}$$

The subtlety worth stating in an interview. Depth is not *entirely* wasted: W is a product of matrices, so if the layers are narrow — say width $r \ll d$ — then $\text{rank}(W) \leq r$, and the network is restricted to low-rank affine maps. That is a constraint, not extra capacity. This is exactly the mechanism LoRA exploits deliberately. The hypothesis class is still linear either way.

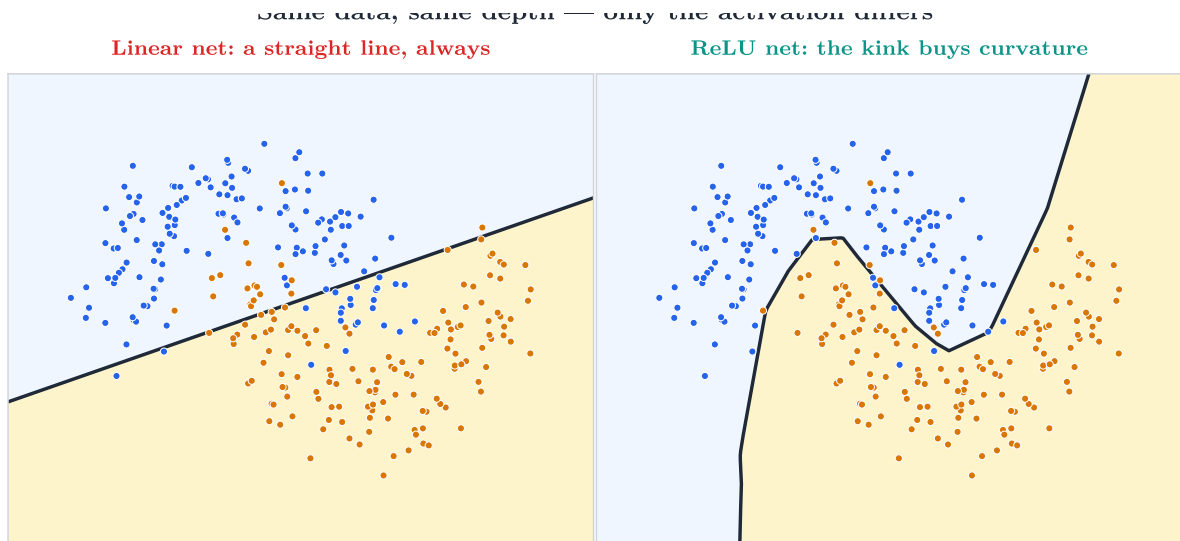


Figure 14.2: Both panels are real networks trained by gradient descent on the same data; the only difference is ReLU. The linear network's boundary is provably a straight line no matter how long you train it — the curvature on the right is bought entirely by the kink.

The nonlinear activation is the “kink” that prevents this collapse. Each kink lets the network bend its decision surface, and stacking many kinks lets it bend into arbitrarily complex shapes — which is what the **Universal Approximation Theorem** guarantees: with enough neurons and at least one nonlinearity, a network can approximate essentially any continuous function on a compact set to any desired accuracy.

What the Universal Approximation Theorem does not say. It is an *existence* result: it promises the weights exist, not that gradient descent will find them, and not that the width required is reasonable. The classical statement (Cybenko, 1989; Hornik, 1991) allows the hidden layer to be exponentially wide in the input dimension. Depth is what makes the approximation efficient: there are functions a depth- k network represents with $O(n)$ neurons that a depth-2 network needs exponentially many to match. When an interviewer asks “if one hidden layer is universal, why go deep?” — that gap is the answer.

WORTH KNOWING — THE COLLAPSE ARGUMENT IS WHY LORA WORKS

The same algebra runs in reverse for profit. LoRA fine-tunes a frozen weight matrix W by learning $W + BA$ where B is $d \times r$ and A is $r \times d$ with $r \ll d$. Those two layers have no nonlinearity between them, so by §14.2 they collapse to a single rank- r matrix — which is precisely the point: you get a low-rank update with a fraction of the parameters, and at inference you can fold BA back into W for zero added latency. The “flaw” of Step 3 is someone else's feature.

INTERVIEW QUESTION — COMMONLY CITED AT GOOGLE & OTHER TOP TECH COMPANIES

“Why do we need nonlinear activation functions? What happens without them?”

This is exactly §14.2: without a nonlinearity, $W^{(2)}(W^{(1)}x) = (W^{(2)}W^{(1)})x$, and by induction any depth collapses to a single affine map $Wx + b$ — you have spent millions of parameters to fit linear regression. The strong follow-up, which is where the derivation's closing note earns its keep: depth still constrains *rank*, so a deep linear net is a low-rank linear model, and gradient descent on it has genuinely different dynamics — but the hypothesis class is unchanged, and that is what matters.

14.3 The activation function's evolution

The choice of which nonlinearity matters enormously, and its history is a story of fixing one problem after another.

Sigmoid and tanh (the old guard). Smooth S-curves. The fatal flaw is *saturation* — at their flat extremes the curve barely changes, so the slope (which the network needs for learning) is nearly zero. In deep networks, multiplying many near-zero slopes together makes the learning signal vanish before it reaches the early layers. That is Chapter 15's whole story, and the bound below is the number that drives it.

DERIVATION — THE SIGMOID DERIVATIVE, AND WHY IT CANNOT EXCEED $\frac{1}{4}$

Differentiate u^{-1} with $u = 1 + e^{-z}$, giving $-u^{-2}u'$, and $u' = -e^{-z}$.

Split the fraction, then note that $e^{-z}/(1 + e^{-z}) = 1 - \sigma(z)$. This identity is why sigmoid backward passes are so cheap: the forward activation is all you need to cache.

Step 1 — write the sigmoid and differentiate by the quotient rule

$$\sigma(z) = (1 + e^{-z})^{-1} \Rightarrow \sigma'(z) = e^{-z} / (1 + e^{-z})^2$$

Write $s = \sigma(z) \in (0, 1)$. The second derivative is negative everywhere, so g is strictly concave and its stationary point is the global maximum — proven, not asserted.

Step 2 — rewrite in terms of σ itself

$$\sigma'(z) = \sigma(z) (1 - \sigma(z))$$

Step 3 — maximize $s(1 - s)$ over the range of σ

$$g(s) = s(1 - s) = s - s^2, \quad g'(s) = 1 - 2s = 0 \Rightarrow s^* = \frac{1}{2}, \quad g''(s) = -2 < 0$$

$$\max_z \sigma'(z) = \frac{1}{4} \text{ attained only at } z = 0$$

The consequence. In the backward chain of §15.3, every sigmoid layer contributes a factor of at most $\frac{1}{4}$. Across L layers the gradient is scaled by at most $(\frac{1}{4})^L$ before the weights even get a say: for $L = 10$ that is a ceiling of $\approx 10^{-6}$. Not “might vanish” — *cannot avoid* shrinking by at least a million-fold. By the same argument $\tanh'(z) = 1 - \tanh^2(z)$ peaks at 1, which is why tanh was preferred over sigmoid for hidden layers long before ReLU arrived.

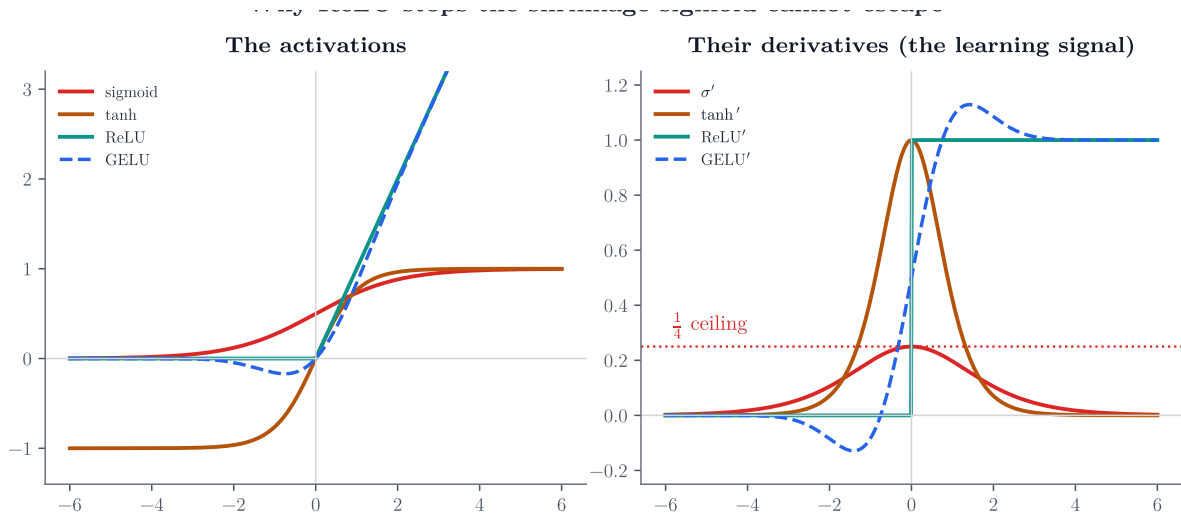


Figure 14.3: Real curves and their exact derivatives. On the right is the $\frac{1}{4}$ bound from the derivation above, drawn: sigmoid's gradient literally cannot reach the dashed line. ReLU's flat slope of 1 across the entire positive axis is why the chain of §15.3 stops shrinking.

ReLU (the breakthrough). $\max(0, z)$ — just “pass positive values through, zero out negatives.” Its slope is exactly 1 for any positive input, so it never saturates on the positive side and the learning signal passes through undiminished. This one change made very deep networks trainable and kicked off the modern era.

ReLU's own flaw is *dying ReLU*: if a neuron is pushed into the negative zone for every input in the data, its slope is 0 there, so it receives no gradient, so its weights never move — dead permanently. This spawned Leaky ReLU (a small negative slope $\alpha \approx 0.01$ so neurons can revive) and the smooth GELU/Swish variants that modern Transformers use. The arc: sigmoid $\rightarrow$ ReLU (fix saturation) $\rightarrow$ Leaky/GELU (fix dying + smoothness).

Activation	$f(z)$	$f'(z)$	The reason it exists
Sigmoid	$1/(1+e^{-z})$	$\sigma(1-\sigma)$, max $\frac{1}{4}$	Historical; still correct for output layers of binary classifiers
Tanh	$(e^z - e^{-z}) / (e^z + e^{-z})$	$1 - \tanh^2$, max 1	Zero-centered, so gradients don't all share a sign; still saturates
ReLU	$\max(0, z)$	1 if $z > 0$, else 0	Kills saturation on the positive side; cheap; sparse activations
Leaky ReLU	$\max(\alpha z, z)$	1 if $z > 0$, else α	A dead neuron can come back, since $\alpha > 0$
GELU	$z \cdot \Phi(z)$	$\Phi(z) + z\phi(z)$	Smooth everywhere; the Transformer default (BERT, GPT)

WORTH KNOWING — GELU IS A STOCHASTIC IDEA IN DISGUISE

GELU is $z \cdot \Phi(z)$, where Φ is the standard normal CDF — so it is the *expected value* of a gate that keeps z with probability $\Phi(z)$ and zeroes it otherwise. That is dropout's coin-flip (§17.2) made deterministic and input-dependent: large inputs are kept almost surely, very negative ones almost never. Its derivative $\Phi(z) + z\phi(z)$ is smooth and briefly exceeds 1 near $z \approx 1$ and dips negative around $z \approx -1$ — both visible in Figure 14.3, and neither is a bug. Swish ($z \cdot \sigma(\beta z)$) is the same idea with a logistic gate.

WORTH KNOWING — DEAD RELUS ARE A LEARNING-RATE SYMPTOM, NOT A CURSE

The usual cause is a large gradient step that drives the bias strongly negative — after which the unit outputs 0 for every input in the data and never recovers. So the first fix is not Leaky ReLU, it is *lower the learning rate* (or use a warmup, §16.7). Diagnose it by logging the fraction of units with zero activation across a validation batch: a healthy ReLU net sits near 50%; a layer at 90%+ that keeps climbing is dying. Note this is a per-dataset property — a unit is only dead if it is inactive for *all* inputs, which is why a sparse-but-alive layer is fine and expected.

WORTH KNOWING — THE MODERN GATED FAMILY: SILU, GELU, AND SWIGLU

Today's large models have mostly converged on *gated* activations, and it helps to see them as one family. SiLU (a.k.a. Swish, $z \cdot \sigma(z)$) and GELU are near-identical smooth curves — both are z multiplied by a soft “keep probability.” The bigger shift is *SwiGLU*, used in Llama, PaLM and most recent LLMs: the feed-forward block splits into two linear projections, one gated by SiLU and multiplied into the other, $(\text{SiLU}(xW_1)) \odot (xW_2)$. It costs a third projection but reliably lowers loss, which is why the extra parameters are considered worth it. If asked “what activation would you use in a new Transformer today,” SwiGLU is the current answer, and naming the gating intuition from the GELU box above is what makes it sound understood rather than memorized.

INTERVIEW QUESTION — COMMONLY CITED AT GOOGLE & OTHER TOP TECH COMPANIES

“Walk me through why ReLU replaced sigmoid in hidden layers.”

Point at the two derivative panels of Figure 14.3 and the $\frac{1}{4}$ bound proven in §14.3. Sigmoid's gradient is at most $\frac{1}{4}$ at its single best point and decays toward 0 at both extremes, so an L -layer stack multiplies the signal by $\leq (\frac{1}{4})^L$ — the vanishing gradient of §15.3 is guaranteed by the function's own algebra. ReLU's derivative is exactly 1 on the positive side, so it contributes no shrinkage at all; it is also cheaper (a comparison, not an exponential) and yields sparse activations. Then volunteer the cost — dead units, non-differentiability at 0 (frameworks just pick a subgradient), and non-zero-centered outputs — plus the fix: Leaky ReLU or GELU.

14.4 Chapter 14 in one breath

CHAPTER 14 IN ONE BREATH

A neuron is linear regression plus a nonlinear kink, and that kink is load-bearing: without it, L layers provably collapse to one affine map $Wx + b$, so depth buys only a rank constraint, not capacity. Activation functions then evolved to keep gradients alive: sigmoid's derivative is capped at exactly $\frac{1}{4}$ — proven by maximizing $s(1-s)$ — which dooms deep stacks to shrink the signal at least four-fold per layer; ReLU's slope of 1 on the positive side removes that shrinkage entirely and made modern depth trainable; Leaky ReLU and GELU then fixed dying units and added smoothness. Modern Transformers use GELU and its gated cousin SwiGLU.

CHAPTER 15

Backpropagation, and the Vanishing Signal

15.1 The blame-assignment problem

A deep network makes a prediction, and it is wrong. Somewhere among millions of weights, some deserve more blame than others for the error. Training means adjusting each weight in proportion to its blame. But how do you figure out how much a weight buried three layers deep contributed to a mistake at the output?

The naive approach — nudge each weight one at a time, see how the error changes — needs one full forward pass per parameter. For P parameters that is $O(P)$ network evaluations per training step; at $P = 10^8$ it is hopeless. **Backpropagation** computes the blame for every weight in a single backward sweep, at a cost of roughly one forward pass. It is the algorithm that makes deep learning computationally possible at all.

The cost, stated precisely. Forward pass: $O(P)$. Backward pass: $O(P)$ — about $2\times$ the forward cost in practice, since each layer computes both $\partial\mathcal{L}/\partial W$ and the δ to pass down. Finite differences: $O(P)$ forward passes, i.e. $O(P^2)$ total. The whole field lives in the gap between $O(P)$ and $O(P^2)$. The price is memory: every intermediate activation must be cached for the backward pass, which is why *activation* memory — not parameter memory — is usually what makes a large model OOM.

WORTH KNOWING — GRADIENT CHECKPOINTING: BUYING MEMORY BACK WITH COMPUTE

The activation-memory cost above is not just trivia — it is the single most common reason a model that *fits* in principle still OOMs in practice, and there is a standard cure worth naming. *Gradient checkpointing* (activation recomputation) stores only a sparse set of activations on the forward pass and recomputes the rest on demand during the backward pass. With checkpoints every $\sqrt{L}$ layers, activation memory drops from $O(L)$ to $O(\sqrt{L})$ at the cost of one extra forward pass — roughly +33% compute for a large memory saving. It is one line in modern frameworks (`torch.utils.checkpoint`, or `gradient_checkpointing=True` in Hugging Face) and is standard for training large models on limited hardware. The trade it makes — recompute rather than store — is the exact mirror of the memoization that makes backprop fast, which is a satisfying thing to point out.

15.2 The chain of responsibility

Backprop is the chain rule from calculus, but the intuition is a *chain of responsibility* running backward through the network. The error at the output depends on the last layer, which depends on the layer before it, which depends on the one before that, all the way back.

The two sweeps that make backprop cheap

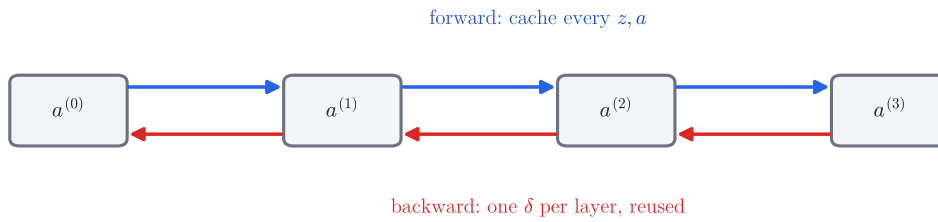


Figure 15.1: The two sweeps. Forward, every z and a is cached; backward, a single δ per layer is computed once and reused by both the weight gradient and the next δ down. That reuse — not the chain rule itself — is what makes backprop fast.

Define the error signal at layer l as $\delta^{(l)} = \partial \mathcal{L} / \partial z^{(l)}$ — how much the loss changes per unit change in that layer's pre-activation. Everything follows from it.

DERIVATION — BACKPROPAGATION, FROM THE CHAIN RULE

Setup: a feedforward net with $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$ and $a^{(l)} = f(z^{(l)})$, for $l = 1 \dots L$, with loss $\mathcal{L}$ computed from $a^{(L)}$. We want $\partial\mathcal{L}/\partial W^{(l)}$ for every l . Because f acts elementwise, the Jacobian $\partial a/\partial z$ is diagonal, and multiplying by a diagonal matrix is exactly the elementwise product $\odot$.

The multivariable chain rule: $z_j^{(l)}$ influences $\mathcal{L}$ only by way of every unit of the next layer, so we sum over all k paths. This sum is the entire content of backprop.

Step 1 — the base case at the output layer

$$\delta^{(L)} = \nabla_a \mathcal{L} \odot f'(z^{(L)})$$

Only the paths through the next layer's pre-activations matter, because f is element-wise: unit i 's pre-activation does not touch unit j 's activation for $i \neq j$.

Step 2 — how a change in $z^{(l)}$ reaches the loss

$$\partial\mathcal{L}/\partial z_j^{(l)} = \sum_k \delta_k^{(l+1)} \cdot (\partial z_k^{(l+1)}/\partial z_j^{(l)})$$

Pull f' out of the sum (it does not depend on k). The remaining sum $\sum_k W_{kj} \delta_k$ is the j -th entry of $W^T \delta$ — the transpose appears because we sum over the *first* index of W .

Step 3 — evaluate the inner partial derivative

$$z_k^{(l+1)} = \sum_i W_{ki}^{(l+1)} f(z_i^{(l)}) + b_k^{(l+1)} \Rightarrow \partial z_k^{(l+1)}/\partial z_j^{(l)} = W_{kj}^{(l+1)} f'(z_j^{(l)})$$

Step 4 — substitute back and recognize a matrix-vector product

$$\delta^{(l)} = \left((W^{(l+1)})^T \delta^{(l+1)} \right) \odot f'(z^{(l)})$$

Since $z_i^{(l)} = \sum_j W_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)}$, we get $\partial z_i/\partial W_{ij} = a_j^{(l-1)}$ directly. The outer-product form is what a GPU actually executes; likewise $\partial\mathcal{L}/\partial b^{(l)} = \delta^{(l)}$.

Step 5 — turn the error signal into the weight gradient

$$\partial\mathcal{L}/\partial W^{(l)} = \delta^{(l)} (a^{(l-1)})^T$$

Read Step 4 again — it is the whole chapter. Going back one layer multiplies δ by $(W^{(l+1)})^T$ and by $f'(z^{(l)})$. Two factors, applied multiplicatively, once per layer. §15.3 is just this observation taken seriously.

The key efficiency: when computing blame for layer 2, backprop reuses the δ it already computed for layer 3 — it does not recompute from scratch. That reuse of intermediate results is exactly *dynamic programming* (the memoization idea from your CS prep). Backprop

is memoization over the network's computation graph. Naming this connection in an interview — “backprop is dynamic programming on the computation graph” — signals real understanding.

WORTH KNOWING — BACKPROP IS REVERSE-MODE AUTODIFF, AND THE DIRECTION IS THE POINT

Backprop is one instance of *reverse-mode* automatic differentiation applied to a computation graph. Reverse mode computes the gradient of one output with respect to many inputs in a single sweep; forward mode does the opposite — one input, many outputs. Deep learning has exactly one scalar output (the loss) and millions of inputs (the weights), so reverse mode wins by a factor of P . That is the entire reason the field uses it. If you ever need a Jacobian-vector product instead of a vector-Jacobian product, forward mode (`jax.jvp`, `torch.func.jvp`) is the cheaper tool.

INTERVIEW QUESTION — COMMONLY CITED AT GOOGLE & OTHER TOP TECH COMPANIES

“Why is backpropagation more efficient than computing numerical gradients?”

This is §15.1 and §15.2 together. Finite differences perturb each parameter separately, costing one forward pass each — $O(P)$ passes, so $O(P^2)$ work, plus floating-point cancellation error. Backprop exploits the structure the naive method throws away: by Step 4 of the derivation, $\delta^{(l)}$ is built from $\delta^{(l+1)}$, so each intermediate is computed once and reused by every weight in that layer — memoization on the computation graph, giving all P gradients for $\sim 2\times$ one forward pass. Finite differences still have a job: gradient-checking a hand-written backward pass on a tiny model.

15.3 Why the signal vanishes

Now the most important idea in this chapter — and it is already sitting in Step 4 above. Unrolling that recursion from the loss back to layer l gives a product of $L-l$ matrices:

$$\delta^{(l)} = \left(\prod_{k=l+1}^L D^{(k-1)} (W^{(k)})^\top \right) \delta^{(L)}, \quad D^{(k)} = \text{diag}(f'(z^{(k)}))$$

The blame reaching layer l is a product of $L-l$ matrices. Multiply many numbers together, and if they are consistently less than 1, the product shrinks exponentially.

DERIVATION — THE VANISHING/EXPLODING CONDITION, MADE PRECISE

Submultiplicativity of the operator norm: $\|AB\| \leq \|A\|\|B\|$, applied repeatedly. Norms of transposes are equal, so the transpose does not matter here.

D is diagonal, so its operator norm is its largest absolute entry — bounded by the activation's steepest slope γ anywhere. For a matrix, the operator norm is the largest singular value $\sigma_{\max}$.

Step 1 — bound the norm of the product by the product of the norms

$$\|\delta^{(l)}\| \leq (\prod_{k=l+1}^L \|D^{(k-1)}\| \cdot \|W^{(k)}\|) \cdot \|\delta^{(L)}\|$$

With every layer sharing roughly the same scale, the product of $L-l$ identical factors is that factor raised to the power $L-l$. The base $\gamma \cdot \sigma_{\max}$ is the only thing that matters.

Step 2 — bound each factor separately

$$\|D^{(k)}\| \leq \gamma \equiv \sup_z |f'(z)|, \quad \|W^{(k)}\| = \sigma_{\max}(W^{(k)})$$

Step 3 — assume homogeneous layers and collect the exponent

$$\|\delta^{(l)}\| \leq (\gamma \sigma_{\max})^{L-l} \|\delta^{(L)}\|$$

$$\gamma \sigma_{\max} < 1 \Rightarrow \text{vanish} \quad \gamma \sigma_{\max} > 1 \Rightarrow \text{explode}$$

Now plug in real numbers. Sigmoid gives $\gamma = 1/4$ (proven in §14.3). To hold $\gamma \cdot \sigma_{\max} \approx 1$ you would need $\sigma_{\max} \approx 4$ in every layer — enormous weights, which push z far from 0, where $|\sigma'|$ collapses well below $1/4$. The two requirements fight each other: sigmoid deep nets are squeezed between vanishing and saturation with no escape. ReLU gives $\gamma = 1$, so the condition reduces to $\sigma_{\max} \approx 1$ — achievable by initialization alone, which is exactly §15.4.

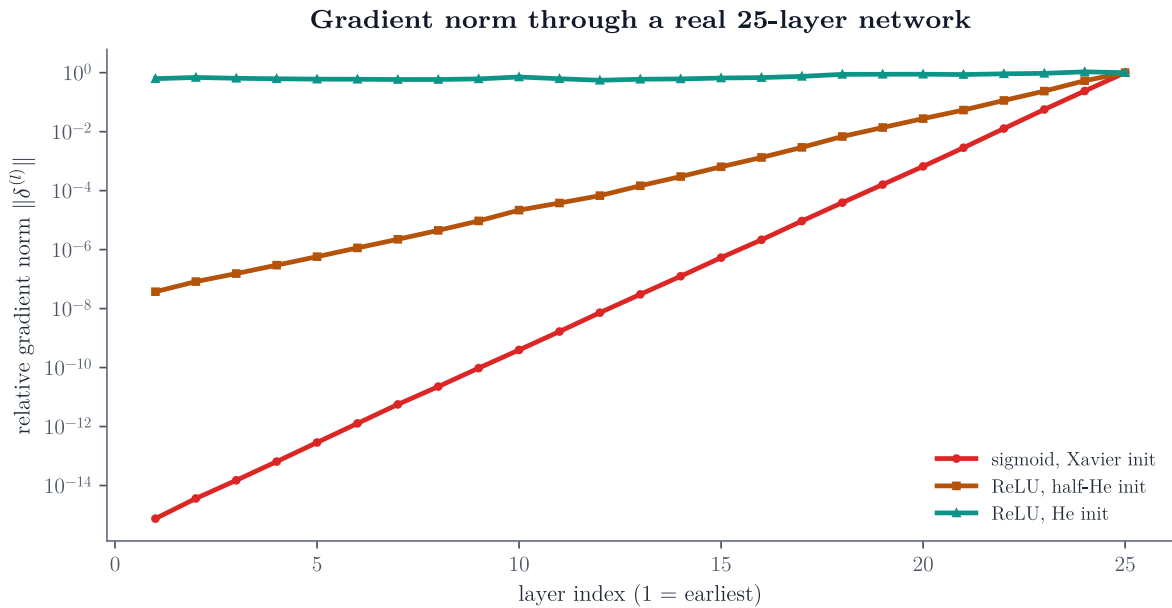


Figure 15.2: Not a cartoon: gradient norms measured through a real 25-layer network. The lines are straight on a log axis, the signature of the exponential decay the derivation predicts. Sigmoid loses many orders of magnitude by layer 1; ReLU with He init sits flat at 1.0, exactly as $\gamma \cdot \sigma_{\max} \approx 1$ requires; the half-He curve shows what even a factor-of-2 init error costs.

By the time the blame reaches the early layers it is essentially zero, so those layers barely update and barely learn. This is the **vanishing gradient problem**, and it is why deep networks were nearly untrainable for decades. The mirror image — factors consistently greater than 1 — gives *exploding gradients*, where the signal blows up and training destabilizes into NaNs.

WORTH KNOWING — EXPLODING GRADIENTS ARE THE EASY HALF

Vanishing is insidious — training simply underperforms, silently. Exploding announces itself: a loss spike, then NaN. It also has a blunt, effective fix that vanishing lacks — *gradient clipping*. Global-norm clipping rescales the whole gradient when it exceeds a threshold: $g \leftarrow g \cdot \min(1, c/\|g\|)$, which preserves the direction and only shortens the step (`torch.nn.utils.clip_grad_norm_`, typical $c \in [0.5, 1.0]$). Prefer it to clipping each coordinate independently, which distorts the direction. Essentially every LLM training run uses it. You cannot symmetrically “un-vanish” a gradient by scaling up, because a vanished signal has lost its information to floating-point underflow — there is nothing left to amplify.

15.4 Initialization: the other half of the fix

The condition $\gamma \cdot \sigma_{\max} \approx 1$ says the activation is only half the story: the weights' scale is the other half. **Initialization** is how you set that scale before training begins, and getting it wrong costs you the network. Figure 15.2's middle line is ReLU — the “good” activation — vanishing anyway, purely because the initial variance was too small.

DERIVATION — HE INITIALIZATION, DERIVED FROM A VARIANCE CONSTRAINT

Goal: choose $\text{Var}(W)$ so that activation variance is preserved layer to layer — no growth, no decay. Assume weights are i.i.d. with mean 0, independent of the inputs, and let n be the layer's fan-in. For independent zero-mean terms, variances add, and $\text{Var}(Wa) = E[W^2]E[a^2] = \text{Var}(W)E[a^2]$. Note it is the second moment $E[a^2]$, not $\text{Var}(a)$ — they differ whenever a has nonzero mean, which ReLU guarantees.

This is the crux. If z is symmetric about 0, then z^2 is identical on both halves, but ReLU zeroes one of them — so exactly half the second moment survives. That factor of $\frac{1}{2}$ is the entire difference between He and Xavier.

Step 1 — variance of a single pre-activation

$$\text{Var}(z_i) = n \cdot \text{Var}(W) \cdot E[a^2]$$

Step 2 — the ReLU-specific step: relate $E[a^2]$ to $\text{Var}(z)$

$$a = \max(0, z) \Rightarrow E[a^2] = \frac{1}{2} E[z^2] = \frac{1}{2} \text{Var}(z)$$

Step 3 — impose variance preservation across the layer

$$\text{Var}(z^{(l)}) = n \cdot \text{Var}(W) \cdot \frac{1}{2} \cdot \text{Var}(z^{(l-1)}) \Rightarrow \text{require } n \cdot \text{Var}(W) \cdot \frac{1}{2} = 1$$

$$\text{Var}(W) = 2/n \quad (\text{He}); \quad \text{Var}(W) = 1/n \quad (\text{Xavier})$$

Reading the result. He init is $W \sim N(0, 2/n_{\text{in}})$: the 2 compensates for the half of the signal ReLU discards. Xavier/Glorot targets $2/(n_{\text{in}} + n_{\text{out}})$, compromising between preserving the forward signal ($1/n_{\text{in}}$) and the backward one ($1/n_{\text{out}}$). Use He with ReLU, Xavier with tanh — `kaiming_normal_` vs `xavier_normal_` in PyTorch. The middle curve of Figure 15.2 is what a factor-of-2 error costs: ReLU with $\text{Var}(W) = 0.5/n$ loses several orders of magnitude over 25 layers, because $(0.5)^{25/2}$ compounds relentlessly.

WORTH KNOWING — MODERN INIT HAS ONE MORE TRICK: START THE RESIDUAL BRANCH AT ZERO

Deep Transformers and ResNets often initialize the last layer of each residual branch to zero (or scale it by $1/\sqrt{L}$). Then every block starts as the exact identity $x \mapsto x + 0$, so at step 0 the network is effectively shallow and perfectly conditioned, and depth “switches on” gradually as the branches learn. Fixup, T-Fixup, and ReZero build on this; GPT-style models scale residual-branch output projections by $1/\sqrt{(2L)}$. It is a nice illustration that init and architecture (§17.4) are solving the same problem from two directions.

INTERVIEW QUESTION — COMMONLY CITED AT GOOGLE & OTHER TOP TECH COMPANIES

“Why can't we just initialize all weights to zero?”

Run it through Step 1 of the He derivation: $\text{Var}(W) = 0$ gives $\text{Var}(z) = 0$, so every unit in a layer outputs the same value. Worse, by Step 4 of the backprop derivation, every unit in a layer then receives an identical δ , so identical weight updates — the units stay clones of each other forever. This is the *symmetry-breaking* problem: a 512-unit layer is permanently a 1-unit layer. Random init breaks the symmetry, and §15.4 is about choosing the right scale once you do — $2/n_{\text{in}}$ for ReLU. Biases can safely start at zero: the weights have already broken the symmetry.

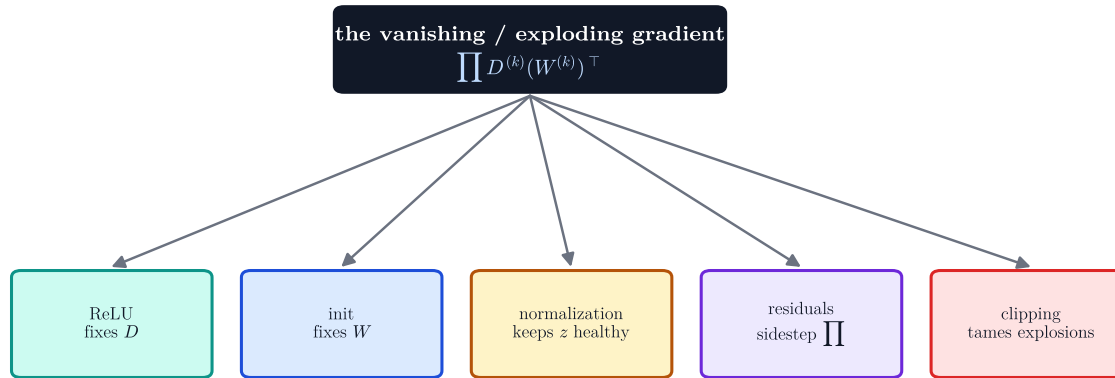
INTERVIEW QUESTION — COMMONLY CITED AT GOOGLE & OTHER TOP TECH COMPANIES

“What is the difference between He and Xavier initialization, and when do you use each?”

This is the payoff of §15.4's derivation, read off Step 3. Both target variance preservation across a layer; they differ by exactly the factor of $\frac{1}{2}$ that ReLU introduces. Xavier/Glorot assumes a roughly linear, symmetric activation ($\tanh$) that passes the full second moment, giving $\text{Var}(W) = 1/n$ (or $2/(n_{\text{in}} + n_{\text{out}})$ to balance the backward pass too). He assumes ReLU, which discards the negative half and so halves the second moment, so you double the weight variance to compensate: $\text{Var}(W) = 2/n$. Rule of thumb: *He with ReLU and its variants, Xavier with tanh/sigmoid*. Getting this wrong by the factor of two is not cosmetic — the middle curve of Figure 15.2 is precisely a factor-of-two error, and it vanishes the gradient by several orders of magnitude over 25 layers.

15.5 The whole zoo of fixes traces back to one problem

An enormous fraction of modern deep learning is, at heart, a response to vanishing gradients. Each fix attacks a different factor in the product from §15.3.



One problem, five families of answer

Figure 15.3: One problem, five families of answer. Read each box as an attack on a specific term of $\prod D^{(k)}(W^{(k)})^\top$: ReLU fixes D , init fixes W , normalization keeps z where D is healthy, residuals sidestep the product entirely, and clipping tames the exploding half.

The recurring insight — which you should be able to state cold — is that *additive paths preserve gradients while multiplicative paths destroy them*. Residual connections and LSTM cell states both exploit this: they give the gradient an additive highway to flow backward on, bypassing the multiplicative shrinkage. Two architectures, eighteen years apart, the same underlying fix.

Why “additive” is the master key, in one line of algebra. A plain layer gives $\partial y / \partial x = F'(x)$, so the backward chain multiplies by F' and dies when $F' \approx 0$. A residual block $y = x + F(x)$ gives $\partial y / \partial x = 1 + F'(x)$. The product over L such blocks contains a term that is identically 1 — the gradient can always get home. §17.4 measures this through 50 layers.

15.6 Chapter 15 in one breath

CHAPTER 15 IN ONE BREATH

Backprop assigns blame to every weight in one backward sweep: the chain rule gives $\delta^{(l)} = ((W^{(l+1)})^T \delta^{(l+1)}) \odot f'(z^{(l)})$, and reusing each δ rather than recomputing it turns an $O(P^2)$ job into an $O(P)$ one — dynamic programming on the computation graph. But that same recursion is multiplicative, so blame reaching layer 1 is scaled by roughly $(\gamma \cdot \sigma_{\max})^{L-1}$: below 1 it vanishes exponentially, above 1 it explodes. Sigmoid's $\gamma = 1/4$ makes vanishing unavoidable; ReLU's $\gamma = 1$ plus He init's $\text{Var}(W) = 2/n$ puts the product back at ≈ 1 . Nearly every modern trick — ReLU, good init, normalization, residuals, LSTM gates, clipping — exists to keep that product near one, and additive paths are the master key.

CHAPTER 16

The Optimizer Family, One Fix at a Time

16.1 A family tree, not a list

The optimizers are not a menu of unrelated choices — they are a lineage, each one fixing a specific flaw in its predecessor. Understanding the family tree lets you derive any of them from the one before, which is exactly the “trace the development” question interviewers love.

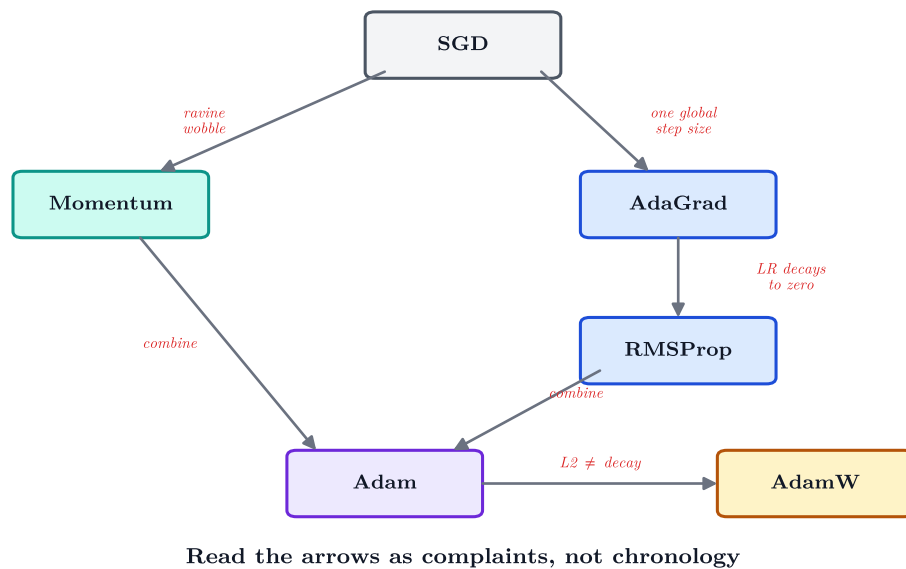


Figure 16.1: Read the arrows as complaints, not chronology. Each optimizer exists because someone identified one concrete failure in the last one — which is why you can reconstruct the whole family from first principles instead of memorizing five update rules.

Every optimizer in this chapter is the same skeleton — a step downhill scaled by something built from the gradient history:

$$\theta_{t+1} = \theta_t - \eta \cdot (\text{a function of } g_1 \dots g_t)$$

16.2 SGD — the hiker with one problem

Plain SGD is Volume I’s blindfolded hiker: feel the slope, step downhill, repeat. Its update is the baseline everything else modifies:

$$g_t = \nabla_{\theta} \mathcal{L}(\theta_t; \mathcal{B}_t), \quad \theta_{t+1} = \theta_t - \eta g_t$$

It works, and its mini-batch noise even helps by shaking the hiker out of sharp minima. But it has one glaring weakness: a single fixed step size suits neither steep nor shallow terrain. In a long narrow valley — a *ravine* — it bounces back and forth across the steep walls while crawling along the gentle floor toward the goal.

DERIVATION — WHY A RAVINE CRIPPLES SGD — THE CONDITION NUMBER

Model the neighborhood of a minimum by a quadratic $\mathcal{L}(\theta) = \frac{1}{2} \theta^\top H \theta$ with H symmetric positive definite, eigenvalues $0 < \lambda_{\min} \leq \dots \leq \lambda_{\max}$. Every smooth loss looks like this close enough to a minimum. H is symmetric, so the spectral theorem gives an orthonormal eigenbasis — in those coordinates the problem is n independent 1-D problems.

Set $u = Q^\top \theta$; then $\mathcal{L} = \frac{1}{2} \sum_i \lambda_i u_i^2$. The coupling was an illusion of the basis.

Step 1 — diagonalize to decouple the axes

$$\mathcal{L}(\theta) = \frac{1}{2} \theta^\top H \theta, \quad H = Q \Lambda Q^\top$$

Each coordinate is multiplied by its own constant every step — geometric dynamics with ratio $r_i = 1 - \eta \lambda_i$.

Step 2 — run one SGD step in each decoupled coordinate

$$g = \nabla \mathcal{L} = \Lambda u \Rightarrow u_i^{(t+1)} = (1 - \eta \lambda_i) u_i^{(t)}$$

A geometric sequence converges iff its ratio is inside the unit interval. The binding constraint is the largest eigenvalue: $\eta < 2/\lambda_{\max}$, or the steep direction diverges.

Step 3 — demand stability on every axis at once

$$u_i^{(t)} = (1 - \eta \lambda_i)^t u_i^{(0)} \rightarrow 0 \Leftrightarrow 0 < \eta < 2/\lambda_i$$

where $\kappa \equiv \lambda_{\max}/\lambda_{\min}$ is the condition number. For $\kappa = 100$ this is ≥ 0.98 : the flat direction shrinks by less than 2% per step, however you tune η .

Step 4 — see what that ceiling costs the flat direction

$$\eta < 2/\lambda_{\max} \Rightarrow |r_{\min}| = |1 - \eta \lambda_{\min}| > |1 - 2/\kappa|$$

$$\kappa \equiv \lambda_{\max}/\lambda_{\min}, \quad \text{steps} \sim \mathcal{O}(\kappa \log(1/\varepsilon))$$

The interview-ready reading. SGD's step size is set by the steepest direction but its progress is set by the flattest one, and κ measures the gap. Deep nets routinely have κ in the thousands. This single quantity explains the rest of the chapter: momentum improves the rate to $\mathcal{O}(\sqrt{\kappa})$ — a quadratic speedup — while AdaGrad/RMSProp/Adam attack κ itself by rescaling each coordinate so the effective spectrum is flatter.

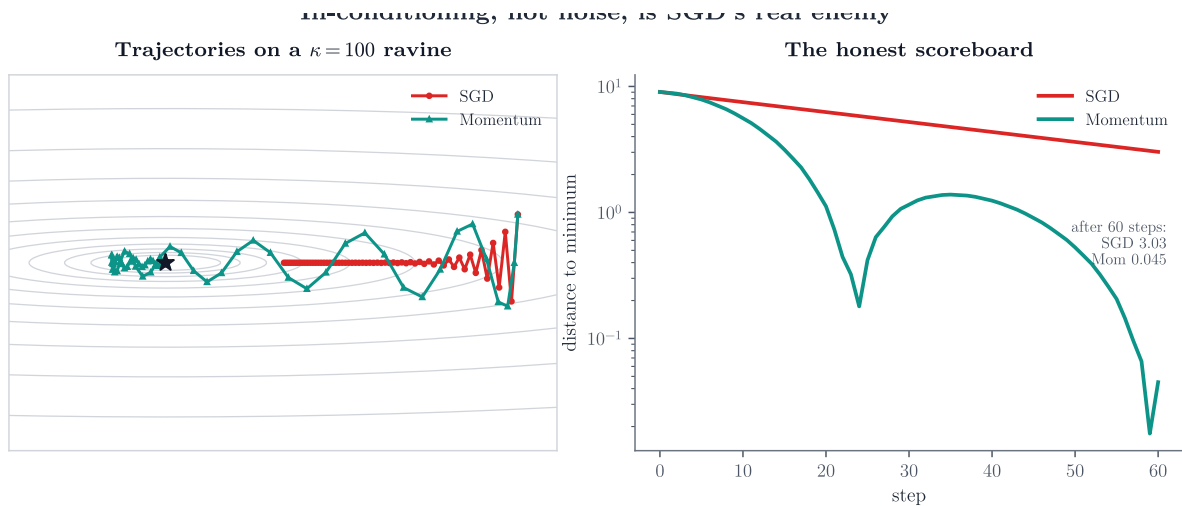


Figure 16.2: A real quadratic with $\kappa = 100$, every method tuned to $\sim 90\%$ of its own stability limit — no strawmen. Left: SGD spends its budget crossing the walls. Right, the honest scoreboard: after 60 steps SGD is still far from the minimum while momentum is an order of magnitude closer, exactly as Step 4 predicts before you run anything.

WORTH KNOWING — THE NOISE IN “STOCHASTIC” IS A FEATURE, AND BATCH SIZE IS ITS DIAL

The gradient on a mini-batch is an unbiased estimate of the full-batch gradient with covariance shrinking like $1/|\mathcal{H}|$. That noise is not merely tolerable — it is thought to bias SGD toward *flat* minima, which tend to generalize better, since a sharp minimum is unstable under gradient noise. This is the leading explanation for the “generalization gap” in large-batch training: bigger batches mean less noise means sharper minima. The practical rule of thumb when you scale the batch is the *linear scaling rule* — multiply the batch by k , multiply η by k , and add a warmup (§16.7) so the first few large steps don't diverge.

INTERVIEW QUESTION — COMMONLY CITED AT GOOGLE & OTHER TOP TECH COMPANIES

“Why does momentum converge in $O(\sqrt{\kappa})$ steps while plain SGD needs $O(\kappa)$?”

This is §16.2's condition-number analysis extended by one idea. SGD's iteration contracts the flat direction by a factor $1 - \eta\lambda_{\min}$, and since η is capped at $2/\lambda_{\max}$, that factor is stuck near $1 - 2/\kappa$ — hence $O(\kappa)$ steps. Momentum turns the first-order recursion into a second-order (heavy-ball) one whose two roots let the slow mode contract like $1 - 2/\sqrt{\kappa}$ when β is tuned to $(\sqrt{\kappa}-1)/(\sqrt{\kappa}+1)$. Square-rooting the condition number is the whole prize, and it is why every serious SGD baseline uses momentum. The intuition to say out loud: momentum averages away the oscillation in the steep direction (§16.3), which lets you spend a larger effective step on the flat one. It is the same $O(\sqrt{\kappa})$ that conjugate gradient achieves on the exact quadratic.

16.3 Momentum — give the hiker inertia

The fix: let the hiker build up momentum, like a ball rolling downhill. Instead of stepping purely by the current slope, accumulate a running average of recent gradients and step by that.

$$v_t = \beta v_{t-1} + g_t, \quad \theta_{t+1} = \theta_t - \eta v_t \quad (\beta \approx 0.9)$$

Why it helps: in the ravine, the side-to-side wobbles point in opposite directions on successive steps, so averaging them cancels the wobble. The forward direction is consistent across steps, so averaging reinforces it. The ball stops bouncing off the walls and accelerates down the valley floor.

DERIVATION — MOMENTUM'S EFFECTIVE STEP, AND THE $1/(1-\beta)$ SPEEDUP

Substitute the recursion into itself repeatedly. Momentum is an exponentially weighted sum of the entire gradient history — the memory decays with a half-life of about 7 steps at $\beta = 0.9$.

A geometric series with ratio $\beta < 1$.

Step 1 — unroll the velocity recursion

$$v_t = \sum_{k=0}^{t-1} \beta^k g_{t-k} \longrightarrow g/(1-\beta)$$

At $\beta = 0.9$ the effective step is $10\times$ the plain SGD step — that is the acceleration, quantified.

Step 2 — the consistent-direction case (the valley floor)

$$g_t \approx g \text{ for all } t \Rightarrow v_t \rightarrow g \cdot \sum_{k=0}^{\infty} \beta^k = g/(1-\beta)$$

Step 3 — the alternating-direction case (the walls)

$$g_t \approx (-1)^t g \Rightarrow v_t \rightarrow g \cdot \sum_k (-\beta)^k = g/(1+\beta)$$

consistent directions amplified $\times 1/(1-\beta) = 10$; oscillating directions damped to $\times 1/(1+\beta) \approx 1/2 \Rightarrow$ a $19\times$ relative preference for the valley floor

The stability caveat behind Figure 16.2. Because the effective step grows by $1/(1-\beta)$, momentum's stable learning rate is smaller: the heavy-ball condition is $\eta(1+\beta) < 2/\lambda_{\max}$. It still wins decisively, because it converges in $O(\sqrt{\kappa})$ steps rather than $O(\kappa)$. *Nesterov* momentum refines this by evaluating the gradient at the look-ahead point $\theta_t - \eta\beta v_{t-1}$, so the ball can brake before it overshoots rather than after.

16.4 AdaGrad — a custom step size per parameter

Momentum still uses one global step size. But some parameters — those attached to frequent features — need small careful steps, while others, tied to rare features like uncommon words in NLP, need big aggressive ones. AdaGrad gives each parameter its own learning rate, shrinking the rate for parameters that have already received large updates.

$$G_t = \sum_{k=1}^t g_k^2, \quad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t} + \varepsilon} \odot g_t$$

Great for sparse data: a parameter that rarely receives a gradient keeps a large effective rate, so when its rare feature finally appears, it learns from it properly. But AdaGrad has a fatal flaw. G_t is a sum of squares, so it is monotonically non-decreasing — every learning rate marches toward zero and can never recover, and training can grind to a halt before reaching the goal.

The failure in one line. If gradients have roughly constant scale g , then $G_t \approx tg^2$, so the effective rate decays like $\eta/(g\sqrt{t})$ — a $1/\sqrt{t}$ schedule you never chose and cannot switch off. That is provably optimal for *convex* problems, which is what AdaGrad was designed for. It is wrong for non-convex deep nets, where the landscape you face at step 10,000 has nothing to do with the gradients from step 10.

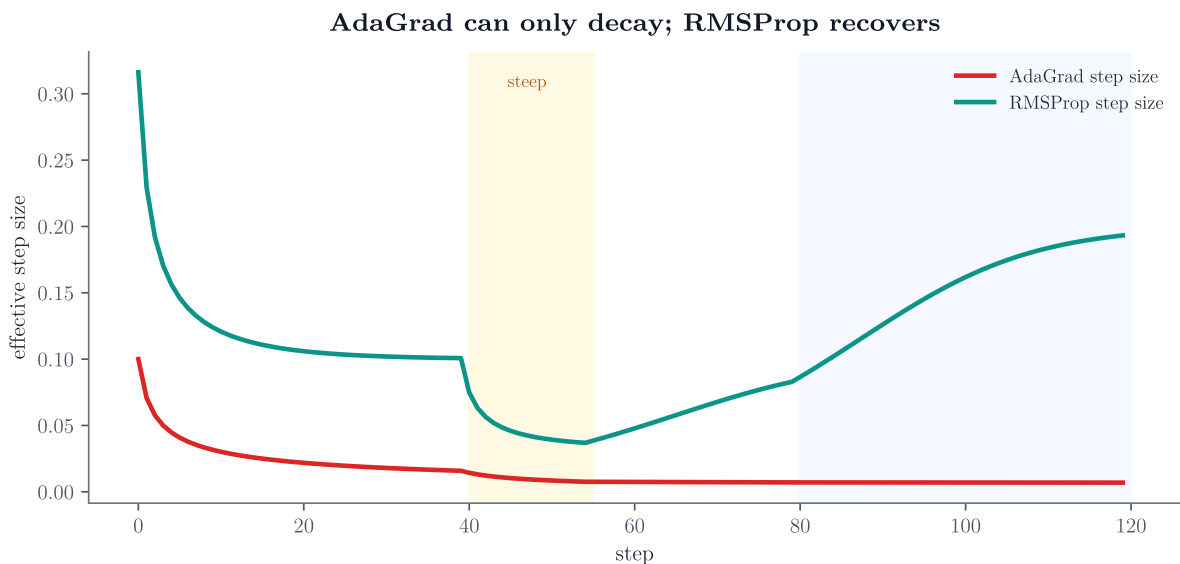


Figure 16.3: The same gradient stream fed to both accumulators. AdaGrad's step size decays monotonically by construction — the curve can never turn back up. RMSProp's decaying average lets the rate rise again when the landscape steepens, which is the whole fix.

16.5 RMSProp — forget the ancient past

The fix for AdaGrad's decay-to-zero: instead of accumulating all past gradients, keep only a decaying average — recent gradients matter, ancient ones fade.

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t} + \varepsilon} \odot g_t$$

One character changes the story: replacing the running sum with a running average makes v_t an estimate of the *current* gradient scale rather than a monument to all past ones. Now the per-parameter rate adapts to the landscape underfoot without collapsing to zero. Same adaptivity, no premature death.

WORTH KNOWING — WHAT DIVIDING BY $\sqrt{v}$ ACTUALLY BUYS: UNIT-FREE STEPS

Look at the dimensions. g has units of $\mathcal{L}$ per θ ; $\sqrt{v}$ has the same units; so $g/\sqrt{v}$ is dimensionless and the step is η times a number of order 1 — regardless of whether that parameter's gradients are of size 10^{-7} or 10^3 . This is why adaptive methods need so little learning-rate tuning, and why $\eta = 10^{-3}$ is a sane Adam default across wildly different architectures while SGD's best η varies by orders of magnitude. It also explains ε 's real job: it is not just anti-division-by-zero, it caps the maximum step at η/ε for a parameter whose gradient has been ~ 0 .

16.6 Adam — combine the two best ideas

Adam = Momentum + RMSProp. It tracks two running averages: the average gradient (momentum's idea — which way to go, smoothed), and the average squared gradient (RMSProp's idea — how big a step per parameter).

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = m_t / (1 - \beta_1^t), \quad \hat{v}_t = v_t / (1 - \beta_2^t)$$

$$\theta_{t+1} = \theta_t - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$$

DERIVATION — BIAS CORRECTION — WHERE $1/(1-\beta^T)$ COMES FROM

Both accumulators start at $m_0 = v_0 = 0$. That zero is not a neutral choice — it is a data point claiming “the gradient is zero,” and it drags the average down for many steps. Here is the exact size of the lie, and its exact antidote.

The m_0 term vanishes, leaving only real gradients — but under-weighted, as the next step shows.

Step 1 — unroll the first-moment recursion

$$m_t = (1-\beta_1) \sum_{k=1}^t \beta_1^{t-k} g_k$$

Assume $E[g_k] \approx E[g]$ is roughly constant over the averaging window, pull it out, and re-index with $j = t-k$.

Step 2 — take expectations under a stationary-gradient assumption

$$E[m_t] = (1-\beta_1) \sum_{k=1}^t \beta_1^{t-k} \cdot E[g] = E[g] \cdot (1-\beta_1) \sum_{j=0}^{t-1} \beta_1^j$$

There is the bias, exactly: m_t estimates not $E[g]$ but $(1-\beta_1^t)$ times it. The identical argument on v_t gives $E[v_t] = (1-\beta_2^t)E[g^2]$.

Step 3 — evaluate the finite geometric sum

$$(1-\beta_1) \cdot (1-\beta_1^t)/(1-\beta_1) = 1 - \beta_1^t \Rightarrow E[m_t] = (1 - \beta_1^t) E[g]$$

Step 4 — divide the bias away

$$\hat{m}_t = m_t/(1-\beta_1^t) = E[g] \quad \checkmark \text{ unbiased at every } t$$

$$\hat{m}_t = m_t/(1-\beta_1^t), \quad \hat{v}_t = v_t/(1-\beta_2^t) \rightarrow 1 \text{ as } t \text{ grows, so it fades on its own}$$

Why it matters most for v . At $t = 1$ with $\beta_2 = 0.999$, $v_1 = 0.001 \cdot g^2$ — a thousand-fold underestimate. Uncorrected, the step would be $m_1/\sqrt{v_1} \approx 3.2$ — over $3\times$ the intended η , in a near-random direction, at the most fragile moment in training. Correction makes both estimates honest from step 1. Figure 16.4 is this arithmetic, measured.

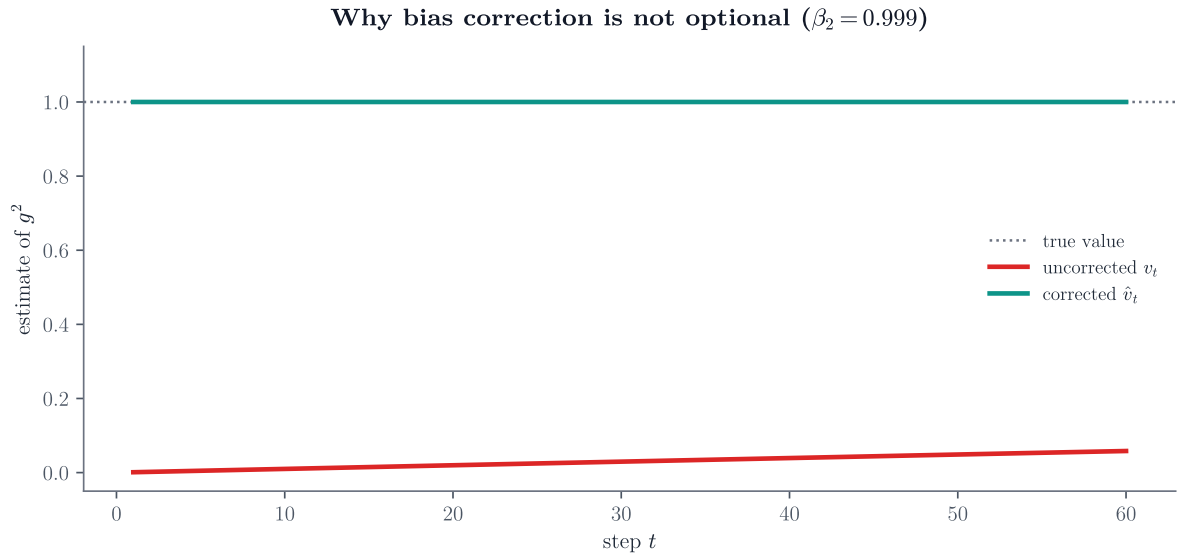


Figure 16.4: A constant gradient of 1 fed to both accumulators. Uncorrected, v_t is still only ~ 0.06 after 60 steps because $\beta_2 = 0.999$ forgets almost nothing per step; the corrected estimate sits exactly on the true value from $t = 1$. The asymmetry between the curves is why bias correction is not optional.

The result inherits momentum's smoothing and RMSProp's per-parameter adaptivity, needs little learning-rate tuning, and converges fast — which is why it is the default optimizer for most deep learning.

The interview-ready sentence. “Adam is momentum's smoothed-direction memory plus RMSProp's per-parameter step-size memory, with bias correction because both start at zero. Each ancestor fixed a specific flaw: momentum fixed SGD's ravine-wobble, AdaGrad added per-parameter rates, RMSProp fixed AdaGrad's decay-to-zero, Adam fused them.”

INTERVIEW QUESTION — COMMONLY CITED AT GOOGLE & OTHER TOP TECH COMPANIES

“What problem does Adam’s bias correction solve, and why does it fade away?”

This is §16.6’s derivation. Both moments start at zero, so by Step 3 $E[m_t] = (1-\beta_1^t)E[g]$ — the estimates are shrunk toward zero early on, severely for v since $\beta_2 = 0.999$ means v_1 is $1000\times$ too small. Dividing by $(1-\beta^t)$ is the exact reciprocal of that factor, so the estimate is unbiased at every step. It fades because $\beta^t \rightarrow 0$, so the divisor $\rightarrow 1$ — by step ~ 1000 it does nothing, which is why its effect is confined to early training. Strong follow-up: this is also why warmup exists — bias correction fixes the expectation but early v is still a high-variance estimate from few samples, so the first steps are unreliable regardless.

WORTH KNOWING — WHEN ADAM IS THE WRONG DEFAULT

Three cases worth naming. **Well-tuned SGD+momentum** often generalizes slightly better on large-scale vision — Adam converges faster but tends toward sharper minima — so ImageNet-style work still uses SGD. **Adam can fail to converge** even on convex problems: the AMSGrad paper exhibited a counterexample where the exponential average lets a rare, informative large gradient be forgotten, and fixed it by forcing v to be non-decreasing. And **memory**: Adam stores m and v per parameter, tripling optimizer state — which is why 8-bit Adam and Adafactor exist for very large models.

16.7 Learning-rate schedules and the modern footnote

No optimizer removes the need to vary η over training. Two schedule ideas dominate modern practice, and both follow directly from what you have already derived.

Warmup. Start η near zero and ramp it up over the first few thousand steps. The justification is §16.6’s follow-up: early v is a high-variance estimate built from a handful of gradients, so early steps are unreliable — and at initialization the loss surface is at its most badly conditioned. Warmup is now standard for every Transformer.

Decay. Then anneal η downward — cosine decay is the common choice. By Step 3 of §16.2, a coordinate converges geometrically to a noise floor set by η and the gradient noise; shrinking η lowers that floor and lets the iterate settle. Large η explores, small η refines.

AdamW (Adam with decoupled weight decay) is now standard for training Transformers and LLMs. It fixes a subtle bug in how Adam interacted with L2 regularization.

DERIVATION — WHY L2 REGULARIZATION BREAKS UNDER ADAM — AND WHAT ADAMW CHANGES

This is the Volume II definition: penalize large weights inside the objective. The decay term $\lambda\theta$ is now part of the gradient — and that is the whole problem.

Adam divides the entire gradient — regularization term included — by $\sqrt{v}$. The penalty gets rescaled per-parameter along with everything else.

Step 1 — write L2 the classical way, as a loss term

$$\mathcal{L}(\theta) = \mathcal{L}(\theta) + (\lambda/2)\|\theta\|^2 \Rightarrow g_t = \nabla\mathcal{L}(\theta_t) + \lambda\theta_t$$

Isolate the $\lambda\theta$ contribution. It is divided by that parameter's own gradient history — so parameters with large, noisy gradients get weakly regularized and quiet parameters get strongly regularized. That is backwards from the intent, and λ 's meaning drifts during training.

Step 2 — push that gradient through Adam's normalization

$$\Delta\theta_t \propto -\eta \cdot (\nabla\mathcal{L} + \lambda\theta_t) / (\sqrt{v_t} + \varepsilon)$$

Step 3 — read off the effective decay per parameter

$$\text{effective decay on } \theta_i = \eta\lambda / (\sqrt{v_{t,i}} + \varepsilon)$$

$$\theta_{t+1} = \theta_t - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon) - \eta\lambda\theta_t$$

The fix in one word: decoupling. AdamW applies weight decay *outside* the adaptive normalization — a separate, uniform shrink of every weight toward zero, independent of v . For plain SGD, L2 and weight decay are algebraically identical, which is why the field conflated them for years; under any adaptive optimizer they are not, and PyTorch's `Adam(weight_decay= $\lambda$ )` gives you the broken Step 2 version while `AdamW(weight_decay= $\lambda$ )` gives the decoupled one. This is the single most cited “subtle optimizer bug” in modern practice — knowing it signals you are current.

WORTH KNOWING — AFTER ADAM: LION, SOPHIA, AND THE MEMORY-LEAN OPTIMIZERS

AdamW is still the default, but two research directions are worth being able to name. *Memory-lean* optimizers attack Adam's tripled state: Adafactor factorizes the second-moment matrix into row/column statistics, and 8-bit Adam quantizes m and v — both let you train larger models on the same hardware. *Sign-based* optimizers attack the update rule itself: *Lion* (discovered by symbolic search) keeps only a momentum buffer and steps by the *sign* of an interpolated momentum, halving optimizer memory versus Adam and often matching it; *Sophia* adds a light-touch Hessian estimate for faster LLM pre-training. The through-line to state in an interview: every one of these is still playing the §16.2 game — cheapen the per-parameter rescaling, or estimate curvature more directly — so the family tree of this chapter is genuinely still growing.

16.8 Chapter 16 in one breath**CHAPTER 16 IN ONE BREATH**

The optimizers are a lineage of fixes, and the condition number $\kappa = \lambda_{\max}/\lambda_{\min}$ is the thread: SGD's step is capped by the steepest direction ($\eta < 2/\lambda_{\max}$) but its progress is set by the flattest, so it needs $O(\kappa)$ steps. Momentum's exponential average amplifies consistent directions by $1/(1-\beta) = 10\times$ and damps oscillating ones to $\sim 1/2$, improving the rate to $O(\sqrt{\kappa})$; AdaGrad adds per-parameter rates for sparse data but its monotone $\sum g^2$ forces an unstoppable $1/\sqrt{t}$ decay; RMSProp swaps that sum for a decaying average; Adam fuses momentum and RMSProp and divides by $(1-\beta^t)$ to undo the exact zero-init bias. AdamW then decouples weight decay from the adaptive rescaling — the modern LLM standard — with warmup and cosine decay on top.

CHAPTER 17

Regularizing Deep Nets, Three Pillars

17.1 Deep nets overfit spectacularly — three defenses

A network with millions of parameters can memorize almost anything, so it overfits readily. Beyond classical weight decay (L2 from Volume II, and see §16.7 for how Adam mangles it), three techniques are the pillars of training deep nets that generalize: *dropout*, *batch normalization*, and *residual connections*. Each has a clean intuition — and, as you will see, two of the three are really also answers to Chapter 15's vanishing-gradient problem.

17.2 Dropout — forcing redundancy

During training, dropout randomly “kills” a fraction of neurons on each step — sets their output to zero. It sounds destructive; it is brilliant.

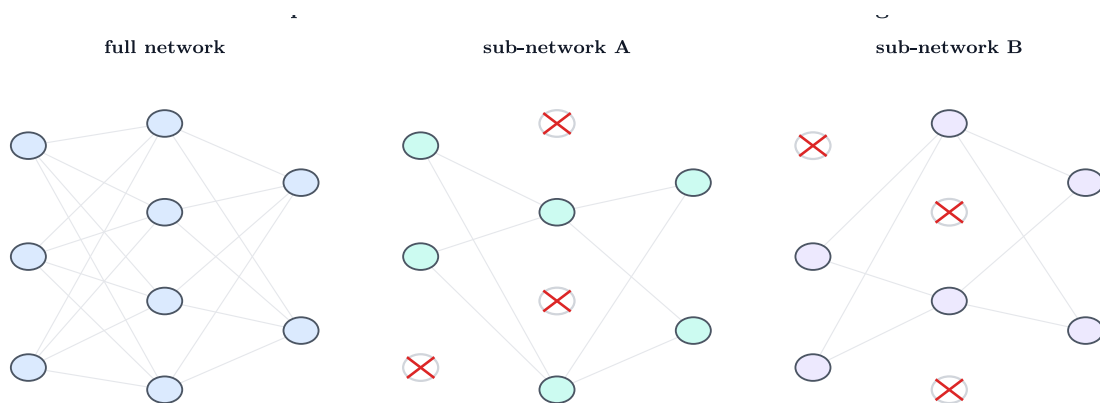


Figure 17.1: Each training step samples a different sub-network from the same weights. With n droppable units there are 2^n such sub-networks sharing parameters — the ensemble of Volume III, hidden inside one model and trained for the price of one.

Why it works — two intuitions. First, it prevents *co-adaptation*: if any neuron might vanish at any moment, no neuron can rely on a specific partner being present, so each must learn a robust, independently-useful feature. Second, it is a *stealth ensemble*: each training step uses a different random sub-network, so you are effectively training an exponential army of overlapping networks and averaging them at test time — bagging (Volume III) hidden inside a single network.

At inference, all neurons stay on. But that changes the expected input to every downstream unit, so something must compensate — and the arithmetic of that compensation is worth doing once.

DERIVATION — INVERTED DROPOUT — WHY THE $1/(1-p)$ FACTOR IS THERE

Let p be the drop probability and a a unit's activation. Training multiplies a by a mask $r \sim \text{Bernoulli}(1-p)$; inference uses no mask. The requirement: a downstream unit should see the same *expected* input either way, or every layer's scale shifts between train and test. A $\text{Bernoulli}(1-p)$ variable has mean $1-p$.

At $p = 0.5$ a unit that learned to expect inputs summing to s suddenly receives $2s$ at test time — enough to push a saturating activation flat, or shift every BatchNorm statistic. The network would silently break.

Step 1 — expected activation during training, naive dropout

$$\mathbb{E}[r \cdot a] = \mathbb{E}[r] a = (1 - p) a$$

Step 2 — the mismatch, stated

train: $(1-p)a$ vs test: $a \Rightarrow$ every downstream pre-activation is inflated by $1/(1-p)$ at test time

Both equalize the expectations. Option (b) is *inverted dropout*, and it is what every framework actually implements — because it keeps inference code identical to a network that never had dropout, so `model.eval()` just switches the mask off.

Step 3 — two ways to repair it

(a) *scale at test: $a \leftarrow (1-p)a$. (b) scale at train: $a \leftarrow ra/(1-p)$*

Step 4 — verify (b) is unbiased

$$\mathbb{E}[ra/(1-p)] = (1-p)a/(1-p) = a \quad \checkmark$$

$$\text{train: } a \leftarrow ra/(1-p); \quad \text{test: } a \leftarrow a$$

The caveat that makes it an approximation. Matching expectations is not the same as averaging the sub-networks' predictions — for a nonlinear network, $\mathbb{E}[f(x)] \neq f(\mathbb{E}[x])$. Test-time dropout is a cheap first-order approximation to the true ensemble average, and it is exact only for linear models. Sampling many masks at test time and averaging — *Monte Carlo dropout* — is the honest version, and it doubles as a practical uncertainty estimate.

WORTH KNOWING — DROPOUT HAS QUIETLY FALLEN OUT OF FAVOR IN THE LARGEST MODELS

Note the interaction: dropout injects noise into a layer's statistics, which is exactly what BatchNorm is trying to estimate stably — the two together can hurt, and the standard advice is to place dropout *after* normalization, or use only one. Modern convolutional nets often drop dropout entirely in favor of BatchNorm plus heavy data augmentation, and large language models use small rates (0.0–0.1) or none at all: when training data vastly exceeds parameters and you make a single pass over it, memorization is not the binding constraint, so the regularizer costs capacity without buying generalization.

INTERVIEW QUESTION — COMMONLY CITED AT GOOGLE & OTHER TOP TECH COMPANIES

“Why does dropout need different behavior at training and test time?”

This is §17.2's derivation. During training a unit's output is ra with $E[r] = 1-p$, so its expected contribution is $(1-p)a$; at test time with all units on it is a . Left unrepaired, every downstream pre-activation is inflated by $1/(1-p)$ — $2\times$ at $p = 0.5$ — and the network breaks. Inverted dropout divides by $(1-p)$ during training so inference needs no special case. The follow-up worth volunteering: this only matches expectations, not the true ensemble average, since $E[f(x)] \neq f(E[x])$ for nonlinear f — MC dropout is the exact-but-expensive version.

17.3 Batch normalization — smoothing the terrain

As training proceeds, the distribution of inputs to each layer keeps shifting, because the earlier layers' weights keep changing. Batch norm normalizes each layer's inputs — recentering and rescaling them to a consistent, healthy range — before passing them on.

$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_i z_i, \quad \sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_i (z_i - \mu_{\mathcal{B}})^2$$

$$\hat{z}_i = \frac{z_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}, \quad y_i = \gamma \hat{z}_i + \beta$$

The learnable γ and β matter more than they look. Without them, BatchNorm would force every layer's pre-activations to be zero-mean and unit-variance — which, for a sigmoid or tanh, pins the unit to the near-linear part of its curve and destroys the expressiveness §14.2 fought for. With them, the network can learn to undo the normalization entirely ($\gamma = \sigma_{\mathcal{H}} \beta = \mu_{\mathcal{H}}$) if that is what it wants. Normalization proposes; the network disposes.

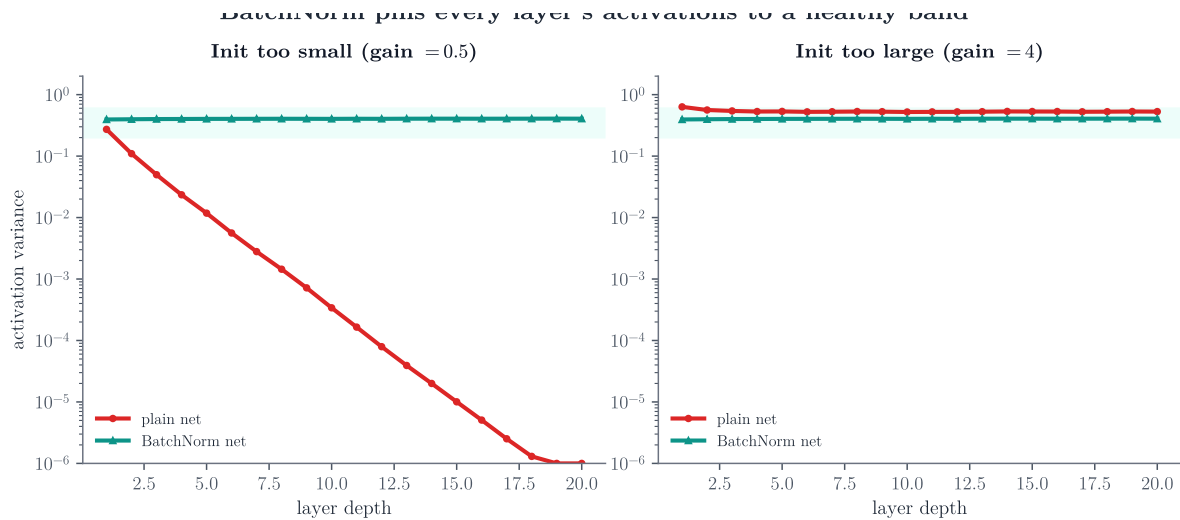


Figure 17.2: The mechanism behind BatchNorm's reputation, measured: activation variance at each depth of a 20-layer tanh network. When the init gain is too small (left) a plain net's variance collapses toward zero; when it is too large (right) it drifts upward — either way the early layers are starved or saturated. BatchNorm pins every layer to the same healthy band regardless of the init, which is what makes the landscape well-conditioned enough to tolerate large learning rates.

The practical payoff: it smooths the loss landscape, which lets you train with much larger learning rates (faster training), makes the network less sensitive to weight initialization, and adds a mild regularizing effect — the batch statistics inject a little noise, since a given example's normalization depends on whichever other examples happened to share its batch.

LayerNorm — a cousin that normalizes across features per example rather than across the batch — is what Transformers use, because it does not depend on batch size and handles variable-length sequences.

The train/test asymmetry is BatchNorm's sharpest edge — and a classic interview trap. At training time μ and σ come from the current mini-batch. At test time there may be no batch (batch size 1), and predictions must not depend on which other examples you happened to send together — so BatchNorm instead uses a *running average* of μ and σ accumulated during training. That is why `model.eval()` is mandatory, and why forgetting it produces the classic bug: great training metrics, garbage inference. It also breaks with very small batches, where the statistics are too noisy — the usual reason to reach for GroupNorm or LayerNorm instead.

WORTH KNOWING — “INTERNAL COVARIATE SHIFT” IS PROBABLY THE WRONG EXPLANATION

The original paper motivated BatchNorm as reducing *internal covariate shift* — the drift in each layer's input distribution as earlier weights update. That story is intuitive and widely repeated, but later work (Santurkar et al., 2018) injected deliberate distributional noise after BatchNorm and found training still improved, and argued the real mechanism is that BatchNorm makes the loss landscape *smoother* — improving its Lipschitz constants — which is what permits the large learning rates. Saying “it reduces internal covariate shift, though that explanation has been challenged; the better-supported one is landscape smoothing” is a genuinely strong interview answer.

WORTH KNOWING — RMSNORM: THE LAYERNORM SUCCESSOR MOST NEW LLMS USE

LayerNorm does two things — recenter (subtract the mean) and rescale (divide by the standard deviation). *RMSNorm* keeps only the second: it divides by the root-mean-square of the activations and drops the mean-subtraction and the β shift entirely, $\hat{x} = x / \text{RMS}(x) \cdot \gamma$. It is cheaper (no mean, one fewer reduction) and, empirically, just as effective — which is why Llama, Mistral, and most recent open LLMs use it in place of LayerNorm. The takeaway to state: the *recentering* half of normalization turned out to contribute little once residual connections and modern init are in place, so the field quietly dropped it. It is a clean example of an idea from this chapter being pared down to its load-bearing part.

INTERVIEW QUESTION — COMMONLY CITED AT GOOGLE & OTHER TOP TECH COMPANIES

“Your model trains beautifully but produces garbage at inference. Where do you look first?”

Name `model.eval()` immediately — and then explain why, which is the point of the box in §17.3. Two layers behave differently in the two modes. BatchNorm switches from mini-batch statistics to its stored running averages; dropout switches its mask off (§17.2). Leave the model in `train()` mode and your predictions depend on whatever else is in the batch, and a batch of size 1 normalizes to exactly zero — hence garbage. The mirror-image bug is calling `eval()` and forgetting to return to `train()`, which silently freezes both.

17.4 Residual connections — the gradient highway

We met this idea in Chapter 15: instead of forcing a layer to transform its input from scratch, let it learn only a small adjustment — a *residual* — added to the input, which passes through unchanged via a skip connection.

One symbol of difference in the backward pass

Plain block: dies when $F' \approx 0$

Residual: the 1 always survives



Figure 17.3: One symbol of difference. The plain block's backward pass multiplies by F' and dies when $F' \approx 0$; the residual block multiplies by $1 + F'$, and that 1 is a term the vanishing-gradient product of §15.3 cannot touch.

DERIVATION — THE SKIP CONNECTION'S GRADIENT, THROUGH L BLOCKS

The derivative of the identity path is the identity matrix — and crucially it is *added*, not multiplied. That single term is the entire mechanism.

Same chain rule as §15.3, but every factor now has an I in it. Compare the plain-network product $\prod D^{(k)}(W^{(k)})^\top$, where nothing prevents a factor from being tiny.

Step 1 — differentiate a single residual block

$$x_{l+1} = x_l + F(x_l, W_l) \Rightarrow \partial x_{l+1} / \partial x_l = I + \partial F / \partial x_l$$

Writing $J_k \equiv \partial F / \partial x_k$, the product expands into every subset of the J's. The subset of size zero contributes exactly I.

Step 2 — chain through L blocks

$$\partial \mathcal{L} / \partial x_l = (\partial \mathcal{L} / \partial x_L) \cdot \prod_{k=l}^{L-1} (I + \partial F / \partial x_k)$$

Step 3 — expand the product and isolate the guaranteed term

$$\prod_k (I + J_k) = I + \sum_k J_k + \sum_{k < j} J_k J_j + \dots$$

$$\frac{\partial \mathcal{L}}{\partial x_l} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{k=l}^{L-1} (I + J_k) = \frac{\partial \mathcal{L}}{\partial x_L} \left(I + \sum_k J_k + \dots \right)$$

Read the guarantee carefully. In a plain network, if the J's are small the gradient is gone: the product of small things is smaller. In a residual network the worst case is the identity — the deep network degenerates gracefully into a shallower one rather than into silence. This also answers “why is it easier to learn a residual?”: if the optimal map for a block is close to identity, F only has to learn ≈ 0 , which is trivial, whereas a plain block would have to learn the identity map exactly — surprisingly hard.

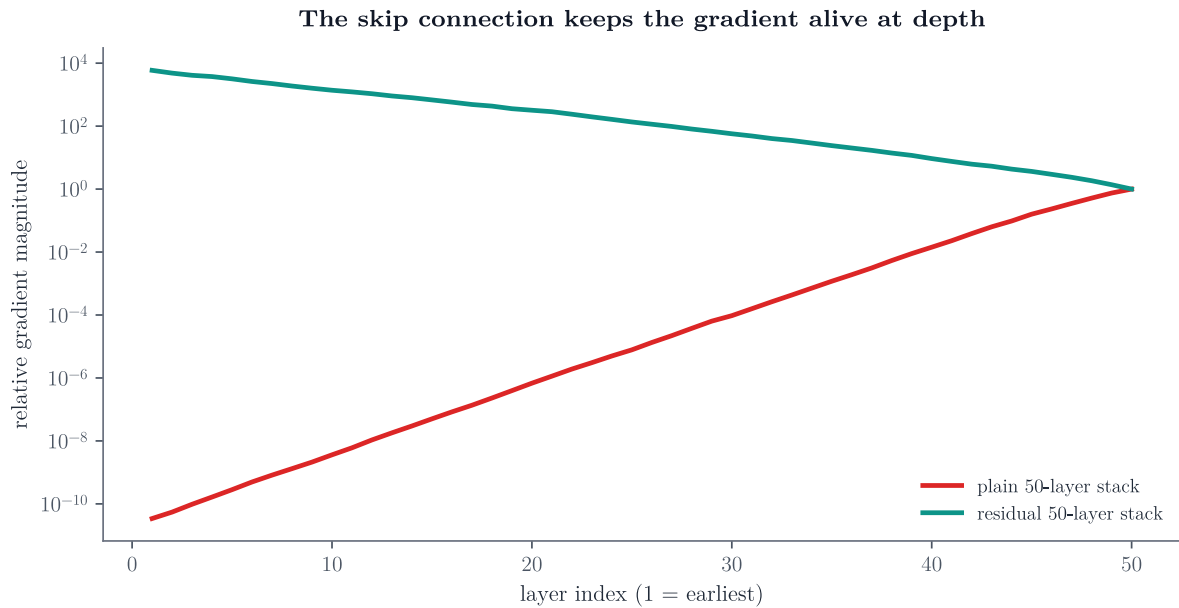


Figure 17.4: Two 50-layer stacks, identical weights and activations, differing only by the skip connections. The plain stack's gradient decays many orders of magnitude before reaching layer 1 — the early layers are effectively frozen. The residual stack's stays flat, exactly as the I term of Step 3 guarantees.

Why it is transformative: during backprop, that $+x$ skip gives the gradient a path with slope exactly 1 — a highway straight back through the network, immune to the multiplicative shrinkage that causes vanishing gradients. This is what let networks go from ~20 layers to 100+ (ResNet), and every Transformer block uses residual connections around both its attention and feedforward sub-layers. It is the same additive-path trick as LSTM's cell state (Volume VI) — the single most important idea for training very deep or very long models.

WORTH KNOWING — PRE-NORM VS POST-NORM: WHERE YOU PUT THE LAYERNORM DECIDES IF THE HIGHWAY IS CLEAN

The original Transformer was *post-norm*: $x \leftarrow \text{LayerNorm}(x + F(x))$. That puts a normalization on the skip path, so the clean I of Step 1 is no longer clean, and deep post-norm stacks need careful warmup to train at all. Modern LLMs use *pre-norm*: $x \leftarrow x + F(\text{LayerNorm}(x))$, which leaves the identity path completely unobstructed from the loss all the way to the embeddings — and trains far more robustly at depth. If you can explain why moving one LayerNorm makes 100-layer Transformers trainable, you have understood this entire chapter.

INTERVIEW QUESTION — COMMONLY CITED AT GOOGLE & OTHER TOP TECH COMPANIES*“How do residual connections help with vanishing gradients?”*

This is §17.4's derivation, and the answer is one term. A plain block gives $\partial y / \partial x = F'$, so L blocks multiply L factors and §15.3's exponential decay applies. A residual block gives $I + \partial F / \partial x$, so expanding the product over L blocks always contains a standalone I — a path with derivative exactly 1 from the loss to any layer. Even if every $F' \rightarrow 0$, the gradient arrives intact; the worst case is a shallower network, not a dead one. Figure 17.4 measures many orders of magnitude more gradient at layer 1 of a 50-layer stack. Then land the general principle: additive paths preserve gradients, multiplicative paths destroy them — the same trick as the LSTM cell state.

17.5 Chapter 17 in one breath**CHAPTER 17 IN ONE BREATH**

Three pillars keep deep nets from overfitting and enable depth. Dropout randomly kills neurons, forcing redundant robust features and acting as a hidden ensemble over 2^n sub-networks — with the $1/(1-p)$ inverted-dropout rescale making train and test expectations match. Batch norm holds each layer's inputs in a healthy range regardless of the init (measured: a plain net's activation variance collapses or drifts with depth while BatchNorm stays pinned), smoothing the landscape so you can train at learning rates that would destabilize a plain net, at the cost of a train/test asymmetry that makes `model.eval()` mandatory; LayerNorm — and its leaner successor RMSNorm — is the batch-independent Transformer variant. Residual connections give gradients an additive highway — the product $\prod(I + J_k)$ always retains a standalone I , so the worst case is a shallower net rather than a dead one — defeating vanishing gradients and enabling 100+ layer networks. The same additive trick as LSTM's cell state.

End of Volume V. Companion to Part V of the ML/DL Handbook.